

SQL Methodology & Queries

Business Analyst Case Study | Dataset: Olist E-Commerce (seller marketplace)

03 · SQL METHODOLOGY & QUERIES

Methodology Overview

All analysis was performed in SQLite against the Olist dataset (orders, order_items, sellers, products tables). Five queries were written, each isolating one segmentation cut. Every query defines a 'problem order' consistently as `order_status IN ('canceled', 'unavailable')`.

Query 1 — Overall Cancellation Rate

Purpose: establish the baseline rate used throughout the report.

```
SELECT
  COUNT(*) AS total_orders,
  SUM(CASE WHEN order_status IN ('canceled', 'unavailable')
        THEN 1 ELSE 0 END) AS problem_orders,
  ROUND(100.0 * SUM(CASE WHEN order_status IN ('canceled', 'unavailable')
        THEN 1 ELSE 0 END) / COUNT(*), 2) AS cancellation_rate_pct
FROM orders;
```

Result: 99,441 total orders; 1,234 problem orders; 1.24% rate.

Query 2 — Rate by Product Category

Purpose: test H1. Joins orders to products via order_items; filters to categories with more than 50 orders to avoid small-sample noise.

```
SELECT
  p.product_category_name AS category,
  COUNT(DISTINCT o.order_id) AS total_orders,
  SUM(CASE WHEN o.order_status IN ('canceled', 'unavailable')
        THEN 1 ELSE 0 END) AS problem_orders,
  ROUND(100.0 * SUM(CASE WHEN o.order_status IN ('canceled', 'unavailable')
        THEN 1 ELSE 0 END) / COUNT(DISTINCT o.order_id), 2) AS cancellation_rate_pct
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
GROUP BY p.product_category_name
HAVING COUNT(DISTINCT o.order_id) > 50
ORDER BY cancellation_rate_pct DESC;
```

Query 3 — Rate by Seller Tenure

Purpose: test H2. Uses a window-function CTE to compute each seller's first order date, then derives tenure (in days) at the time of each subsequent order.

```
WITH seller_first_order AS (
  SELECT oi.seller_id, MIN(o.order_purchase_timestamp) AS first_order_date
  FROM order_items oi
  JOIN orders o ON oi.order_id = o.order_id
  GROUP BY oi.seller_id
),
order_with_tenure AS (
  SELECT
```

```

        o.order_id,
        o.order_status,
        CAST(julianday(o.order_purchase_timestamp)
              - julianday(sfo.first_order_date) AS INTEGER) AS days_since_seller_start
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN seller_first_order sfo ON oi.seller_id = sfo.seller_id
)
SELECT
    CASE
        WHEN days_since_seller_start < 30 THEN '0-30 days (new)'
        WHEN days_since_seller_start < 90 THEN '30-90 days'
        ELSE '90+ days (established)'
    END AS seller_tenure_bucket,
    COUNT(*) AS total_orders,
    SUM(CASE WHEN order_status IN ('canceled', 'unavailable')
            THEN 1 ELSE 0 END) AS problem_orders,
    ROUND(100.0 * SUM(CASE WHEN order_status IN ('canceled', 'unavailable')
                        THEN 1 ELSE 0 END) / COUNT(*), 2) AS cancellation_rate_pct
FROM order_with_tenure
GROUP BY seller_tenure_bucket
ORDER BY cancellation_rate_pct DESC;

```

Note: this query operates at the order-item level (112,650 rows), since seller tenure is a seller-level attribute and a single order can include items from multiple sellers.

Query 4 — Rate by Delivery Delay

Purpose: test H3. Compares actual vs. estimated delivery date for delivered orders.

```
SELECT
  CASE
    WHEN julianday(order_delivered_customer_date)
      - julianday(order_estimated_delivery_date) < 0 THEN 'early'
    WHEN julianday(order_delivered_customer_date)
      - julianday(order_estimated_delivery_date) = 0 THEN 'on_time'
    ELSE 'late'
  END AS delivery_bucket,
  COUNT(*) AS total_orders,
  SUM(CASE WHEN order_status IN ('canceled', 'unavailable')
    THEN 1 ELSE 0 END) AS problem_orders,
  ROUND(100.0 * SUM(CASE WHEN order_status IN ('canceled', 'unavailable')
    THEN 1 ELSE 0 END) / COUNT(*), 2) AS cancellation_rate_pct
FROM orders
WHERE order_delivered_customer_date IS NOT NULL
GROUP BY delivery_bucket
ORDER BY cancellation_rate_pct DESC;
```

Limitation: the WHERE clause required for this query (order_delivered_customer_date IS NOT NULL) excludes almost all cancelled/unavailable orders by construction, since those orders rarely reach delivery. This is why H3 was ruled out rather than accepted or rejected on the numbers alone.

Query 5 — Priority Table (Category × Seller Tenure)

Purpose: combine category and seller tenure to (a) confirm H2 holds within categories, and (b) rank segments by an impact score (volume × rate) to prioritize action.

```
WITH seller_first_order AS (
  SELECT oi.seller_id, MIN(o.order_purchase_timestamp) AS first_order_date
  FROM order_items oi JOIN orders o ON oi.order_id = o.order_id
  GROUP BY oi.seller_id
),
combined AS (
  SELECT
    p.product_category_name AS category,
    CASE
      WHEN julianday(o.order_purchase_timestamp)
        - julianday(sfo.first_order_date) < 30 THEN 'new_seller'
      ELSE 'established_seller'
    END AS seller_type,
    o.order_id,
    o.order_status
  FROM orders o
  JOIN order_items oi ON o.order_id = oi.order_id
  JOIN products p ON oi.product_id = p.product_id
  JOIN seller_first_order sfo ON oi.seller_id = sfo.seller_id
)
SELECT
  category,
  seller_type,
  COUNT(DISTINCT order_id) AS order_volume,
  ROUND(100.0 * SUM(CASE WHEN order_status IN ('canceled', 'unavailable')
    THEN 1 ELSE 0 END) / COUNT(DISTINCT order_id), 2) AS cancel_rate_pct,
  ROUND(COUNT(DISTINCT order_id) *
    (100.0 * SUM(CASE WHEN order_status IN ('canceled', 'unavailable')
      THEN 1 ELSE 0 END) / COUNT(DISTINCT order_id)), 0) AS impact_score
```

```
FROM combined
GROUP BY category, seller_type
HAVING order_volume > 30
ORDER BY impact_score DESC
LIMIT 15;
```

Query Index

Query	Tests	Key Output
1. Overall rate	Baseline	1.24% overall cancellation rate
2. By category	H1	Category-level rates, filtered to n > 50
3. By seller tenure	H2	1.08% (new) vs 0.40% (established)
4. By delivery delay	H3	Ruled out — structural data limitation
5. Priority table	H2 (within-category) + prioritization	automotivo / new_seller: 2.51%